Javascript

Introduction, data types, variables, type Coercion. Scope

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Javascript</title>
  </head>
  <body>
    <h1>Javascript</h1>

    <script>
      // implicit coersion
      console.log("2" + 2);
      console.log("2" - "1");
      console.log("3" * 3);
      //explicit coersion
      console.log(Number("123"));
      console.log(parseInt("123px"));

      console.log(String(123));
      console.log((123).toString());

      console.log(Boolean(0));
      //   var fiit = "Academy";
      //   fiit = 100;
      //   fiit = true;
      //   fiit = "a";
      //   fiit = [1, 2, 3];
      //   fiit = { name: "Ganesh" };
      //   fiit = 100n;
      //   console.log(typeof fiit);
      //   var a = null ;
      //   console.log(typeof a);

      let arun = "developer";
      // var -> allows redeclaration , reinitialization
      //   scope => function scope, global scope

      function redmi() {
        var model = "redmi 9 ";
        console.log(arun);
      }

      //   console.log(model);
```

```
      redmi();

      //   var arun = "consulting";
      //   re-initilization
      //   let => block scope
      {
        const block = "blocked scope";
      }
      // console.log(block);
      arun = "singer";
      console.log(arun);

      let kumari = "developer";
      // let => don't allow declaration
      kumari = "consulting";
      console.log(kumari);

      const PI = 3.14;
      //   const => doesn't allow redeclare and reinitiLie
      console.log(PI);

      // javascript -> dynamic typed language
      // data types -> let, var, const

      // Number
      // String
      // Boolean
      // Object

      // 1995 -> brenden eich
      // livescript -> javascript

      // java -> programming language
      // javscript -> scripting language

      // runs in broswer
      // chrome -> v8
      // firefox -> spidermonkey
      //   console.log("Javascript");
      //   document.write("Hello browser");
      // cd => change directory
    </script>
  </body>
</html>
```

Theory Notes

1. Type Coercion

- **Implicit coercion** → JavaScript automatically converts types.
  - `"2" + 2` → `"22"` (string concatenation).
  - `"2" - "1"` → `1` (strings converted to numbers).
  - `"3" * 3` → `9`.
- **Explicit coercion** → developer manually converts types.
  - `Number("123")` → `123`.
  - `parseInt("123px")` → `123`.
  - `String(123)` or `(123).toString()` → `"123"`.
  - `Boolean(0)` → `false`.

2. Variables

- **var**:
  - Function-scoped or globally scoped.
  - Allows redeclaration and reinitialization.
- **let**:
  - Block-scoped.
  - Allows reinitialization but not redeclaration in the same scope.
- **const**:
  - Block-scoped.
  - Cannot be redeclared or reinitialized.

3. Scope

- **Global scope** → accessible everywhere.
- **Function scope** → variables declared with `var` inside a function are local to that function.
- **Block scope** → variables declared with `let` or `const` inside `{}` are only accessible inside that block.

4. Data Types

- **Primitive types**: Number, String, Boolean, Null, Undefined, BigInt, Symbol.
- **Objects**: Arrays, functions, objects.
- JavaScript is **dynamically typed** → variable types can change at runtime.

5. History

- Created in **1995** by Brendan Eich.
- Originally called **LiveScript**, later renamed to JavaScript.
- Runs in browsers using engines:
  - Chrome → V8.

- Firefox → SpiderMonkey.

Practice Tasks

Short Answer

1. What is the difference between implicit and explicit coercion?
2. Why is `var` considered less safe compared to `let` and `const`?
3. What does `Boolean("")` return?
4. What is the difference between == and === in JavaScript?

Coding Exercises

1. Demonstrate implicit coercion with addition, subtraction, and multiplication.
2. Write a function that converts a string `"456"` into a number using both `Number()` and `parseInt()`.
3. Show the difference between `var`, `let`, and `const` in terms of redeclaration and scope.
4. Create a block of code where a `let` variable is accessible only inside `{}`.
5. returns the type of any variable using `typeof`.